

UNIVERSIDAD PRIVADA DOMINGO SAVIO



INFORME FINAL

UPDS JUDGE

Plataforma web para concursos de programación competitiva

Análisis, diseño, implementación y validación

Integrantes

Wilson Yucra Rengifo
Cristhian Joel Amador Gallardo
Arnold Daniel Torrez Zarate
Daniel Javier Aramayo Mancilla
Enny Anaí Lopez Saldaña Beymar
Angelo Vasquez Acha
Alex Saul Fernandez Valdez

29 de julio de 2026

Revisión histórica

Versión	Hito documental	Fecha
0.1	Versión inicial del informe con diagnóstico, MVP, backlog, UML y arquitectura propuesta.	Sin fecha documentada
0.2	Reconciliación con la auditoría documental aprobada y el estado vigente del frontend.	28/07/2026
0.3	Incorporación de los requerimientos iniciales RF-01 a RF-27 y RNF-01 a RNF-19.	29/07/2026
0.4	Incorporación de planificación, matriz RACI y gestión de riesgos.	29/07/2026
1.0	Versión final compilada para revisión manual del responsable del proyecto.	29/07/2026
1.1	Incorporación de guía de despliegue, entrevista, actores, recursos digitales y README operativo.	29/07/2026

Cuadro 1: Revisión histórica del informe.

Resumen ejecutivo

UPDS JUDGE es una plataforma académica para organizar concursos de programación competitiva, administrar identidades y permisos, publicar problemas, recibir soluciones, delegar su evaluación a un juez en línea y presentar veredictos, historial y ranking. El producto final integra un frontend React con contratos HTTP de un backend ASP.NET Core y un motor de evaluación documentado. La descripción de backend, persistencia y juez se basa en la documentación técnica final aprobada por el equipo; dichos componentes y sus contratos se consideran correctos para este informe.

El proyecto entregó los flujos centrales de registro, autenticación, roles, concursos públicos y privados, inscripción, problemas, descarga del conjunto de problemas, envío de soluciones, consulta de resultados, historial, ranking, congelamiento, globos, upsolving y administración. La actualización del historial y del ranking se realiza mediante HTTP y polling; no existe un cliente SignalR o WebSocket conectado en el frontend. La principal desviación funcional es **RF-02 — Gestión de perfiles**, cuyo estado final es **NO IMPLEMENTADO**. Por la misma ausencia, **RF-22 — Participación del usuario** se clasifica como **IMPLEMENTADO PARCIALMENTE**, aun cuando sus restantes subcapacidades fueron entregadas.

El documento integra la especificación de requisitos de software (SRS), la arquitectura, el modelado, la gestión Scrum, la planificación, los riesgos, la validación y una matriz de trazabilidad de 46 requerimientos iniciales. Las propuestas de versiones estables de lenguajes y actualización de datos personales, registradas en la retrospectiva del Sprint 3, permanecen como trabajo futuro no implementado.

Índice

Revisión histórica	I
Resumen ejecutivo	I
Índice de figuras	V
Índice de tablas	VI
 I Fundamentos y alcance	 1
1. Introducción y objetivos	1
1.1. Propósito	1
1.2. Antecedentes	1
1.3. Problema identificado	1
1.4. Alcance	1
1.5. Objetivo general	2
1.6. Objetivos específicos	2
1.7. Product Goal	2
1.8. Organización del documento	2
2. Toma de requerimientos	2
2.1. Técnicas utilizadas	2
2.1.1. Entrevista	2
2.2. Actores	3
2.3. Stakeholders	3
2.4. Glosario inicial	3
2.5. Supuestos y dependencias	3
2.6. Restricciones	3
 II Especificación funcional	 4
3. Descripción general del producto	4
3.1. Perspectiva	4
3.2. Características principales	4
3.3. Clases de usuario	4
3.4. Interfaces externas	4
3.5. Interfaces de usuario	5
4. Requerimientos	5
4.1. Requerimientos funcionales iniciales	5
4.2. Requerimientos no funcionales iniciales	5
4.3. Estado final de cumplimiento	6
4.4. Desviaciones	6
4.5. Gestión de perfiles no implementada	6
4.6. Desglose de RF-22	7
4.7. Matriz de trazabilidad	7

5. Casos de uso y modelado funcional	11
5.1. Diagrama general	11
5.2. Casos principales	11
5.3. Flujos básicos y alternativos	12
5.4. Relación entre requerimientos e historias	12
 III Diseño de la solución	 12
6. Arquitectura	12
6.1. Visión general	12
6.2. Arquitectura frontend	13
6.3. Backend	14
6.4. Motor de evaluación	14
6.5. Comunicación entre componentes	14
6.6. Seguridad y roles	14
 7. Modelado	 14
7.1. Diagrama de clases	15
7.2. Modelo de contexto	15
7.3. Diagramas de secuencia o actividad	16
7.4. Modelo de datos	16
7.5. Diccionario de datos o referencia	17
 IV Gestión ágil	 17
8. Metodología Scrum	17
8.1. Organización del equipo	18
8.2. Roles Scrum	18
8.3. Matriz RACI	18
8.4. Product Backlog	20
8.5. Product Goal	20
8.6. Sprint Backlogs y Sprint Goals	20
 9. Ejecución de Sprints	 20
9.1. Sprint 0	20
9.2. Sprint 1	21
9.3. Sprint 2	21
9.4. Sprint 3	21
9.5. Sprint Reviews	21
9.6. Retrospectivas	21
9.7. Acciones futuras no implementadas	21
 V Planificación y control	 21
10. Planificación temporal	21
10.1. Gantt general por Sprint	21
10.2. Gantt de historias de usuario	24
10.3. Hitos y entregables	26

11. Gestión de riesgos	26
11.1. Metodología	26
11.2. Registro priorizado	26
11.3. Matriz de probabilidad e impacto	28
11.4. Medidas de mitigación	30
11.5. Riesgos residuales	30
 VI Implementación, validación y cierre	 30
12. Implementación final	30
12.1. Tecnologías	30
12.2. Módulos principales	30
12.3. Autenticación y autorización	30
12.4. Concursos	30
12.5. Problemas	30
12.6. Envíos	31
12.7. Ranking	31
12.8. Administración	31
12.9. Integración frontend	31
12.10. Despliegue y reproducibilidad	31
12.11. Recursos y evidencias digitales del proyecto	31
 13. Calidad y validación	 32
13.1. Estrategia de pruebas	32
13.2. Pruebas automatizadas	32
13.3. Validaciones manuales	32
13.4. Incidencias y hotfixes	32
13.5. Limitaciones	33
13.6. Trazabilidad final	33
 14. Resultados	 33
14.1. Objetivos alcanzados	33
14.2. Requerimientos cumplidos	33
14.3. Requerimientos parciales	33
14.4. Requerimientos no implementados	33
14.5. Trabajo futuro	33
 15. Conclusiones y recomendaciones	 34
A. Glosario	34
B. Matriz de trazabilidad ampliada	35
C. Historias de usuario	35
D. Diagramas	35
E. Tablas de planificación	36
F. Gestión de riesgos	36
G. Referencias documentales	36

Índice de figuras

1.	Árbol de problemas de UPDS JUDGE. Fuente: documentación inicial aprobada del proyecto.	1
2.	Árbol de soluciones de UPDS JUDGE. Fuente: documentación inicial aprobada del proyecto.	4
3.	Diagrama general de casos de uso. Fuente: fuente PlantUML aprobada del proyecto.	11
4.	Arquitectura de componentes de UPDS JUDGE. Fuente: fuente PlantUML aprobada del proyecto.	13
5.	Secuencia de envío y evaluación de una solución. Fuente: fuente PlantUML aprobada del proyecto.	14
6.	Diagrama de clases de persistencia. Fuente: fuente PlantUML aprobada del proyecto.	15
7.	Modelo de contexto técnico y actores. Fuente: fuente PlantUML aprobada del proyecto.	16
8.	Secuencia de importación de problemas y casos desde ZIP. Fuente: fuente PlantUML aprobada del proyecto.	16
9.	Modelo relacional de UPDS JUDGE. Fuente: fuente PlantUML aprobada del proyecto.	17
10.	Matriz RACI del proyecto. Fuente: elaboración del equipo del proyecto.	19
11.	Diagrama de Gantt general por Sprint. Fuente: elaboración del equipo del proyecto.	23
12.	Distribución temporal de historias de usuario. Fuente: elaboración del equipo del proyecto.	25
13.	Listado priorizado de riesgos del proyecto. Fuente: elaboración del equipo del proyecto.	27
14.	Matriz de probabilidad, impacto y prioridad. Fuente: elaboración del equipo del proyecto.	29

Índice de cuadros

1.	Revisión histórica del informe.	I
2.	Requerimientos funcionales iniciales.	5
3.	Requerimientos no funcionales iniciales.	6
4.	Desglose funcional de RF-22.	7
5.	Matriz canónica de trazabilidad de requerimientos iniciales.	8
6.	Resumen del Product Backlog y estado final.	20
7.	Recursos y evidencias digitales del proyecto.	32

Fase I

Fundamentos y alcance

1. Introducción y objetivos

1.1. Propósito

El propósito de este informe es consolidar, en una estructura académica coherente, la evolución y el resultado final de UPDS JUDGE. Se integran la intención inicial, las historias de usuario, el estado implementado, los hotfixes, las limitaciones y el trabajo futuro, sin confundir planificación histórica con funcionalidad vigente.

1.2. Antecedentes

La programación competitiva requiere coordinar participantes, concursos, problemas, lenguajes, envíos, casos de prueba, veredictos y posiciones. La documentación inicial identificó que el uso de herramientas separadas elevaba la carga operativa y dificultaba conservar trazabilidad. El proyecto se planteó como una plataforma web con frontend y backend separados, persistencia en PostgreSQL y un motor especializado de evaluación.

1.3. Problema identificado

La comunidad académica necesitaba una plataforma integrada para organizar concursos, evaluar automáticamente las soluciones y publicar resultados oportunos. La fragmentación del proceso producía riesgo de inconsistencias, demora en los veredictos y dificultad para mantener un historial y ranking confiables.

La Figura 1.3 sintetiza las causas operativas, técnicas y de gestión que dieron origen al proyecto.

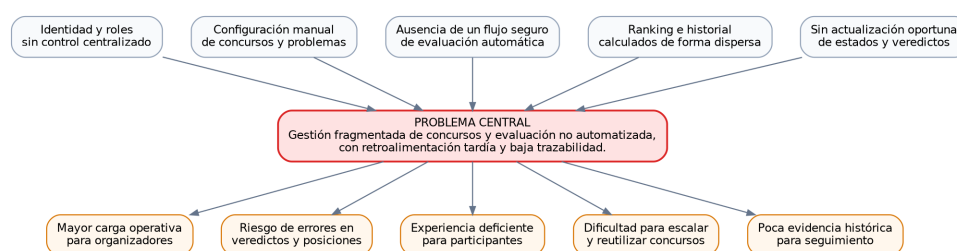


Figura 1: Árbol de problemas de UPDS JUDGE. Fuente: documentación inicial aprobada del proyecto.

1.4. Alcance

El alcance incluye identidad y autorización por roles; administración y participación en concursos; importación y configuración de problemas; recepción y evaluación de soluciones; veredictos; historial; ranking; congelamiento; globos; upsolving; y navegación administrativa de Acceso de Usuario. Quedan fuera del producto entregado la gestión editable del perfil, el cliente push en

tiempo real, la ampliación contractual del catálogo de lenguajes, la detección de plagio y un editor colaborativo.

1.5. Objetivo general

Diseñar e implementar un MVP de UPDS JUDGE que permita administrar concursos de programación, recibir y evaluar soluciones de forma automatizada, consultar veredictos y calcular rankings trazables mediante un frontend React con Vite y un backend ASP.NET Core MVC/Web API separado.

1.6. Objetivos específicos

1. Implementar autenticación y autorización por roles.
2. Permitir la creación, edición y consulta de concursos, incluida la importación segura de problemas.
3. Integrar el flujo de evaluación para C++, Python y C#.
4. Conservar un historial de envíos con estado, veredicto, tiempo y memoria.
5. Calcular ranking, penalización y congelamiento sin incorporar upsolving al resultado oficial.
6. Proporcionar interfaces consistentes, responsivas y accesibles para usuarios y administradores.
7. Mantener trazabilidad entre requerimientos, historias, changes, evidencias y resultados.

1.7. Product Goal

Centralizar el ciclo de una competencia académica —desde la configuración hasta el resultado— para reducir trabajo manual, aplicar reglas uniformes y ofrecer a participantes y organizadores información trazable.

1.8. Organización del documento

El informe se organiza en seis fases: fundamentos y alcance; especificación funcional; diseño; gestión ágil; planificación y control; e implementación, validación y cierre. Los anexos reúnen glosario, referencias documentales y orientación sobre evidencias sin duplicar la matriz canónica de trazabilidad.

2. Toma de requerimientos

2.1. Técnicas utilizadas

La toma y consolidación de requerimientos combinó análisis del problema y de actores, entrevista, revisión de necesidades académicas, historias de usuario, backlog, modelado UML, prototipos visuales documentados, criterios de aceptación, retrospectivas de Sprint y validación documental. Para el cierre se utilizó la auditoría documental aprobada como fuente prioritaria del estado vigente. No se atribuyen al proceso encuestas, talleres ni reuniones no documentadas.

2.1.1. Entrevista

La entrevista tuvo como propósito comprender el contexto académico y competitivo, identificar actores y validar necesidades administrativas y de participación. La síntesis autorizada por el responsable relaciona esta técnica con administración de roles, organización y privacidad de concursos, carga de problemas y casos de prueba, participación estudiantil, envíos, ranking y control de resultados. La entrevista contribuyó a identificar y validar estas necesidades; no se afirma

que cada requerimiento haya sido pronunciado literalmente en el audio ni se utiliza la entrevista como evidencia de implementación.

El recurso autorizado es [Audio de la entrevista de levantamiento de requerimientos](#). Para elaborar este informe no fue necesario descargar, analizar ni transcribir el audio; la información funcional utilizada corresponde a la síntesis suministrada por el responsable.

2.2. Actores

Administrador de Roles — docente o líder de club.

Usuario con el mayor nivel de privilegios en la gestión de identidades y roles. Consulta y gestiona cuentas dentro de los permisos documentados, asigna el rol Administrador de Concursos a docentes o líderes, administra roles y contribuye a la integridad global. Esto no implica autorización para cualquier operación funcional.

Administrador de Concursos — docente u organizador.

Responsable de la dimensión académica y competitiva. Crea y edita concursos; configura modalidad pública o privada, información y duración; incorpora el set de problemas en PDF; administra problemas y casos de prueba en ZIP; gestiona participantes y visibilidad de resultados cuando el contrato lo permite; y consulta herramientas administrativas.

Usuario — estudiante o participante.

Se registra e inicia sesión; consulta e ingresa a concursos según sus reglas; descarga el PDF; consulta problemas; selecciona lenguajes permitidos; envía soluciones; consulta veredictos, historial y ranking; y realiza upsolving cuando corresponde. No dispone de edición de información personal.

Los actores funcionales y los roles de autorización se describen desde las capacidades del sistema. No deben confundirse con roles Scrum ni con responsabilidades de la matriz RACI, que corresponden al trabajo del equipo del proyecto.

2.3. Stakeholders

Además de los actores funcionales, son stakeholders el equipo de desarrollo, el evaluador académico y los responsables de operación de backend y juez. El motor de evaluación participa como sistema externo, no como rol humano.

2.4. Glosario inicial

Los términos principales se desarrollan en el Anexo A. En el contexto funcional, *veredicto* es el resultado normalizado de un envío; *upsolving* es la práctica posterior que no modifica el ranking oficial; y *congelamiento* es el corte temporal que oculta resultados recientes en la vista pública.

2.5. Supuestos y dependencias

Se consideran correctos el backend, el juez, los contratos consumidos y los diagramas aprobados. La operación depende de los servicios HTTP, de la persistencia y de la disponibilidad del motor de evaluación. La descripción no implica una nueva auditoría de dichos componentes.

2.6. Restricciones

El frontend y el backend permanecen separados; el navegador no ejecuta soluciones ni accede directamente al juez; los casos de prueba son privados; y el catálogo inicial se limita a C++ (.cpp), C# (.cs) y Python (.py). La propuesta de ampliar versiones o lenguajes no forma parte del producto entregado.

Fase II

Especificación funcional

3. Descripción general del producto

3.1. Perspectiva

UPDS JUDGE es una aplicación web de tipo SPA (Single Page Application) que consume contratos HTTP/JSON. El backend aplica reglas, autorización, persistencia y coordinación con el juez. La Figura 3.1 resume la respuesta propuesta al problema inicial.

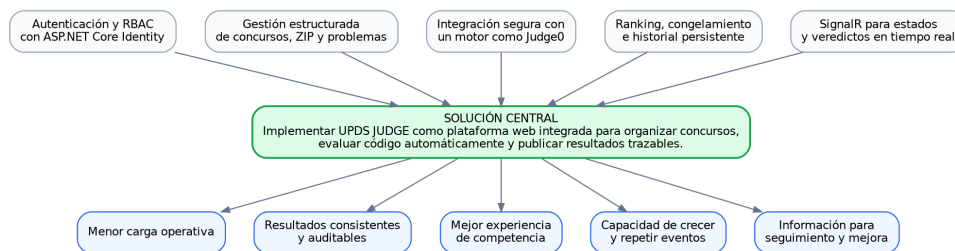


Figura 2: Árbol de soluciones de UPDS JUDGE. Fuente: documentación inicial aprobada del proyecto.

3.2. Características principales

Las características finales incluyen autenticación y sesión, guards de navegación, layouts por rol, concursos, inscripción pública y privada, problemas, PDF, envíos, veredictos, historial global y contextual, ranking, congelamiento, paginación, globos, edición de concursos y administración de roles.

3.3. Clases de usuario

El **Usuario**, estudiante o participante, opera bajo rutas `/student` para concursos, problemas, envíos, historial, ranking y upsolving; no cuenta con una pantalla de perfil editable. El **Administrador de Concursos**, docente u organizador, utiliza rutas de administración y Acceso de Usuario para configurar concursos y consultar sus herramientas. El **Administrador de Roles**, docente o líder de club, utiliza `/admin/roles` para consultar usuarios y administrar asignaciones permitidas. Los guards frontend controlan navegación y experiencia de usuario, mientras la autorización definitiva corresponde al backend.

3.4. Interfaces externas

Las interfaces externas principales son la API HTTP del backend, la persistencia PostgreSQL y el motor de evaluación. El frontend centraliza endpoints, transporte autenticado y tipos contractuales. La actualización del ranking usa polling de diez segundos; el historial se vuelve a consultar tras un envío.

3.5. Interfaces de usuario

La interfaz dispone de plantillas de autenticación, `UserLayout`, `AdminLayout`, sidebar administrativo, encabezado contextual de concurso, formularios, tablas, filtros, paginación, mensajes de carga/vacío/error y componentes de veredictos y globos.

4. Requerimientos

4.1. Requerimientos funcionales iniciales

La Tabla 2 conserva los 27 requerimientos iniciales sin reformularlos para acomodarlos al resultado final.

Cuadro 2: Requerimientos funcionales iniciales.

ID	Requerimiento inicial
RF-01	Registro e inicio de sesión.
RF-02	Gestión de perfiles: visualizar y editar información básica del perfil del usuario.
RF-03	Listado de concursos.
RF-04	Visualización del concurso.
RF-05	Participación en concursos.
RF-06	Listado de problemas.
RF-07	Descarga del conjunto de problemas.
RF-08	Envío de soluciones.
RF-09	Selección de lenguaje.
RF-10	Evaluación automática.
RF-11	Verificación de casos de prueba.
RF-12	Resultado del envío.
RF-13	Historial de envíos.
RF-14	Ranking del concurso.
RF-15	Administración de problemas.
RF-16	Administración de casos de prueba.
RF-17	Gestión de usuarios.
RF-18	Upsolving.
RF-19	Consulta de resultados.
RF-20	Administración de roles.
RF-21	Administración de concursos.
RF-22	Participación del usuario.
RF-23	Control del estado del concurso.
RF-24	Acceso a concursos.
RF-25	Validación de archivos.
RF-26	Consulta del estado de los envíos.
RF-27	Gestión de participantes.

4.2. Requerimientos no funcionales iniciales

Cuadro 3: Requerimientos no funcionales iniciales.

ID	Requerimiento inicial
RNF-01	Rendimiento.
RNF-02	Escalabilidad.
RNF-03	Disponibilidad.
RNF-04	Seguridad.
RNF-05	Integridad de la evaluación.
RNF-06	Aislamiento de ejecución.
RNF-07	Compatibilidad.
RNF-08	Usabilidad.
RNF-09	Mantenibilidad.
RNF-10	Portabilidad.
RNF-11	Persistencia.
RNF-12	Disponibilidad del motor de evaluación.
RNF-13	Auditoría.
RNF-14	Extensibilidad.
RNF-15	Concurrencia.
RNF-16	Stack tecnológico backend.
RNF-17	Stack tecnológico frontend.
RNF-18	Gestión de base de datos.
RNF-19	Arquitectura de evaluación.

4.3. Estado final de cumplimiento

El resultado general confirma la entrega del flujo principal, con limitaciones explícitas en actualización push, mediciones operativas y disponibilidad de dependencias externas. Los estados utilizados son **IMPLEMENTADO**, **IMPLEMENTADO CON LIMITACIONES**, **IMPLEMENTADO PARCIALMENTE** y **NO IMPLEMENTADO**. El detalle canónico aparece en la matriz de la Sección 4.7.

4.4. Desviaciones

Las desviaciones principales son la ausencia de gestión editable del perfil y la falta de push en tiempo real desde el frontend. La primera afecta RF-02 y una subcapacidad de RF-22; la segunda limita RF-12 y RF-26 sin eliminar la consulta funcional por HTTP. Las métricas de rendimiento, disponibilidad y concurrencia no se presentan como garantías cuantificadas si la documentación no registra una medición final.

4.5. Gestión de perfiles no implementada

RF-02 — Gestión de perfiles. Estado final: NO IMPLEMENTADO. El requerimiento pedía visualizar y editar la información básica del perfil del usuario. El producto entregado no contiene pantalla editable de perfil, página de configuración personal, formulario de actualización de datos, flujo de guardado ni actualización de sesión derivada de una edición del perfil. La retrospectiva del Sprint 3 registró la actualización de datos personales como propuesta futura, no implementada y pendiente de priorización.

4.6. Desglose de RF-22

La Tabla 4 explica por qué el estado global de RF-22 es parcial.

ID	Subcapacidad	Estado final
RF-22.a	Consultar concursos	IMPLEMENTADO
RF-22.b	Visualizar un concurso	IMPLEMENTADO
RF-22.c	Descargar el PDF	IMPLEMENTADO
RF-22.d	Enviar soluciones	IMPLEMENTADO
RF-22.e	Consultar estados de envíos	IMPLEMENTADO CON LIMITACIONES
RF-22.f	Consultar el ranking	IMPLEMENTADO
RF-22.g	Consultar historial de envíos	IMPLEMENTADO
RF-22.h	Editar información del perfil	NO IMPLEMENTADO

Cuadro 4: Desglose funcional de RF-22.

Por tanto, **RF-22 — Participación del usuario** tiene estado global **IMPLEMENTADO PARCIALMENTE**.

4.7. Matriz de trazabilidad

La matriz completa se presenta una sola vez. La evidencia combina la auditoría documental, las historias reconciliadas, los changes y hotfixes, las retrospectivas, los diagramas y los contratos consumidos. Para backend y juez se aplica la suposición aprobada de corrección.

Cuadro 5: Matriz canónica de trazabilidad de requerimientos iniciales.

ID	Requerimiento inicial	Tipo	Estado final	Evidencia documental	Historia o change relacionado	Observaciones
RF-01	Registro e inicio de sesión	RF	IMPLEMENTADO	Auditoría: rutas <code>/register</code> , <code>/login</code> , sesión y guards	UJ-05, UJ-06	Registro, token, identidad y cierre de sesión integrados.
RF-02	Gestión de perfiles	RF	NO IMPLEMENTADO	Auditoría y retrospectiva Sprint 3	Sin historia implementada	No existe pantalla, formulario, servicio ni guardado de perfil editable.
RF-03	Listado de concursos	RF	IMPLEMENTADO	Dashboard de usuario, filtros y paginación	UJ-11	Concursos próximos, activos y finalizados.
RF-04	Visualización del concurso	RF	IMPLEMENTADO	Rutas contextuales <code>ContestContextHeader</code> y	UJ-11, UJ-13	Contexto reutilizado para problemas, envíos y ranking.
RF-05	Participación en concursos	RF	IMPLEMENTADO	Política de acceso y endpoint de inscripción	UJ-12	Incluye públicos y privados, incluso privado finalizado según contrato aprobado.
RF-06	Listado de problemas	RF	IMPLEMENTADO	Feature <code>problems</code> y dashboard contractual	UJ-13	Incisos, títulos, límites y estado personal.
RF-07	Descarga del conjunto de problemas	RF	IMPLEMENTADO	Botón de PDF documentado y ruta contextual	UJ-13	Abre el recurso configurado sin bloquear el resto de la pantalla.
RF-08	Envío de soluciones	RF	IMPLEMENTADO	Formulario, editor, carga y POST de envíos	UJ-14	El código se transmite como texto; no se ejecuta en el navegador.
RF-09	Selección de lenguaje	RF	IMPLEMENTADO	Selector C++ 17, Python 3 y C#	UJ-14	Catálogo final no se presenta como ampliado.
RF-10	Evaluación automática	RF	IMPLEMENTADO	Arquitectura y contratos de evaluación aprobados	UJ-14, UJ-15	Backend y juez se consideran correctos.
RF-11	Verificación de casos de prueba	RF	IMPLEMENTADO	Modelado de casos privados y evaluación	UJ-09, UJ-10, UJ-15	Casos protegidos y coordinados por backend/juez.
RF-12	Resultado del envío	RF	IMPLEMENTADO CON LIMITACIONES	Historial y <code>VerdictBadge</code> por HTTP	UJ-15	Resultado visible; no existe push SignalR/WebSocket conectado.
RF-13	Historial de envíos	RF	IMPLEMENTADO	Historial global y contextual, filtros y paginación	UJ-20	Vistas de usuario y Acceso de Usuario.
RF-14	Ranking del concurso	RF	IMPLEMENTADO	Ranking contractual, polling, paginación y hotfix	UJ-18; fix de regresión 18	Frontend conserva puestos y penalización del backend.
RF-15	Administración de problemas	RF	IMPLEMENTADO	Creación/edición de concurso y lista de problemas	UJ-08, UJ-09, UJ-10, UJ-19	Configuración de inciso, límites, memoria y globos.
RF-16	Administración de casos de prueba	RF	IMPLEMENTADO	Importación ZIP y modelo de almacenamiento privado	UJ-09, UJ-10	Validación y persistencia coordinadas por backend aprobado.

Continuación de la Tabla 5

ID	Requerimiento inicial	Tipo	Estado final	Evidencia documental	Historia o change relacionado	Observaciones
RF-17	Gestión de usuarios	RF	IMPLEMENTADO	Consulta, búsqueda y roles de usuarios	UJ-07	Capacidad del Administrador de Roles, acotada a permisos; no equivale a perfil personal.
RF-18	Upsolving	RF	IMPLEMENTADO	Backlog, contratos y ranking que excluye práctica	UJ-16, UJ-18	Los envíos posteriores no alteran el ranking oficial.
RF-19	Consulta de resultados	RF	IMPLEMENTADO	Envíos contextuales e historial global	UJ-15, UJ-20	Veredictos, tiempo y memoria cuando están disponibles.
RF-20	Administración de roles	RF	IMPLEMENTADO	Ruta <code>/admin/roles</code> , búsqueda, asignación y remoción	UJ-07	Administrador de Roles: docente o líder de club, dentro de permisos definidos.
RF-21	Administración de concursos	RF	IMPLEMENTADO	Listado, filtros, resumen, creación y edición	UJ-08, UJ-09, UJ-19	Administrador de Concursos: docente u organizador; incluye ZIP, congelamiento y globos.
RF-22	Participación del usuario	RF	IMPLEMENTADO PARCIALMENTE	Auditoría, UJ-11 a UJ-20 y Tabla 4	UJ-11–UJ-20	Usuario estudiante o participante; todas las subcapacidades principales salvo edición de perfil.
RF-23	Control del estado del concurso	RF	IMPLEMENTADO	Fechas, duración, estados y congelamiento	UJ-08, UJ-11, UJ-19	El backend es fuente de verdad del estado.
RF-24	Acceso a concursos	RF	IMPLEMENTADO	Guards, política de acceso e inscripción	UJ-11, UJ-12	Acceso por sesión, rol, modalidad y participación.
RF-25	Validación de archivos	RF	IMPLEMENTADO	ZIP, extensiones <code>.cpp/.py/.cs</code> y límite de tamaño	UJ-09, UJ-14, UJ-19	Validación cliente y servidor; casos permanecen privados.
RF-26	Consulta del estado de los envíos	RF	IMPLEMENTADO CON LIMITACIONES	Refetch HTTP, botón Actualizar e historial	UJ-15, UJ-20	Sin cliente push en tiempo real.
RF-27	Gestión de participantes	RF	IMPLEMENTADO	Inscripción, participación única y ranking	UJ-12, UJ-18	Administración y participación según permisos; contratos externos asumidos correctos.
RNF-01	Rendimiento	RNF	IMPLEMENTADO CON LIMITACIONES	Polling acotado, paginación y consultas por dominio	UJ-18, UJ-20	No se documenta una garantía cuantitativa final de carga.
RNF-02	Escalabilidad	RNF	IMPLEMENTADO CON LIMITACIONES	Separación frontend/backend, servicios y paginación	Arquitectura aprobada	Arquitectura escalable; capacidad no medida en este cierre.
RNF-03	Disponibilidad	RNF	IMPLEMENTADO CON LIMITACIONES	Recuperación HTTP y manejo de errores	UJ-15	Depende de API, base de datos y juez.
RNF-04	Seguridad	RNF	IMPLEMENTADO	JWT, roles, guards, casos privados y validación	UJ-05–UJ-09	La autorización definitiva reside en backend.

Continuación de la Tabla 5

ID	Requerimiento inicial	Tipo	Estado final	Evidencia documental	Historia o change relacionado	Observaciones
RNF-05	Integridad de la evaluación	RNF	IMPLEMENTADO	Persistencia de envíos, veredictos y resultados por caso	UJ-14, UJ-15	Backend y juez aprobados como fuente de verdad.
RNF-06	Aislamiento de ejecución	RNF	IMPLEMENTADO	Juez externo y navegador sin ejecución de código	Arquitectura de evaluación	El frontend nunca ejecuta soluciones.
RNF-07	Compatibilidad	RNF	IMPLEMENTADO	SPA web y catálogo C++, Python, C#	UJ-14	No implica ampliación futura de lenguajes.
RNF-08	Usabilidad	RNF	IMPLEMENTADO	Responsive, estados, feedback, filtros y accesibilidad	Historias reconciliadas	Layouts y componentes consistentes por rol.
RNF-09	Mantenibilidad	RNF	IMPLEMENTADO	Organización por features, tipos, mappers y tests	Changes de Sprints 1–3	Contratos y componentes compartidos reducen duplicación.
RNF-10	Portabilidad	RNF	IMPLEMENTADO CON LIMITACIONES	Cliente web y separación por servicios	Sprint 0; arquitectura aprobada	La documentación no registra una matriz final de plataformas.
RNF-11	Persistencia	RNF	IMPLEMENTADO	PostgreSQL/EF Core y contratos de historial	Arquitectura de datos	Se asume correcta la implementación aprobada.
RNF-12	Disponibilidad del motor de evaluación	RNF	IMPLEMENTADO CON LIMITACIONES	Manejo de 503 y recuperación de estado	UJ-15	Dependencia externa operativa, sin SLA documentado.
RNF-13	Auditoría	RNF	IMPLEMENTADO	Historial de envíos y auditoría de roles documentada	UJ-07, UJ-20	Registra acciones y resultados relevantes.
RNF-14	Extensibilidad	RNF	IMPLEMENTADO	Features, servicios, catálogo y adaptadores	Arquitectura aprobada	Extensibilidad arquitectónica no equivale a haber ampliado lenguajes.
RNF-15	Concurrencia	RNF	IMPLEMENTADO CON LIMITACIONES	Evaluación externa, estados, polling y consultas	UJ-15, UJ-18	No se consigna prueba de carga final.
RNF-16	Stack tecnológico backend	RNF	IMPLEMENTADO	ASP.NET Core MVC/Web API, EF Core	Documentación técnica aprobada	Backend asumido correcto; no fue reauditado.
RNF-17	Stack tecnológico frontend	RNF	IMPLEMENTADO	React, TypeScript, Vite, Tailwind CSS	README y <code>package.json</code>	Stack verificado en el repositorio frontend.
RNF-18	Gestión de base de datos	RNF	IMPLEMENTADO	PostgreSQL, Identity y modelo relacional	Arquitectura de datos	Persistencia descrita desde documentación aprobada.
RNF-19	Arquitectura de evaluación	RNF	IMPLEMENTADO	Backend encapsula juez, casos y resultados	Diagramas y contratos	El navegador no accede al juez ni a casos privados.

5. Casos de uso y modelado funcional

5.1. Diagrama general

La Figura 5.1 presenta los casos de uso aprobados y su relación con los actores.

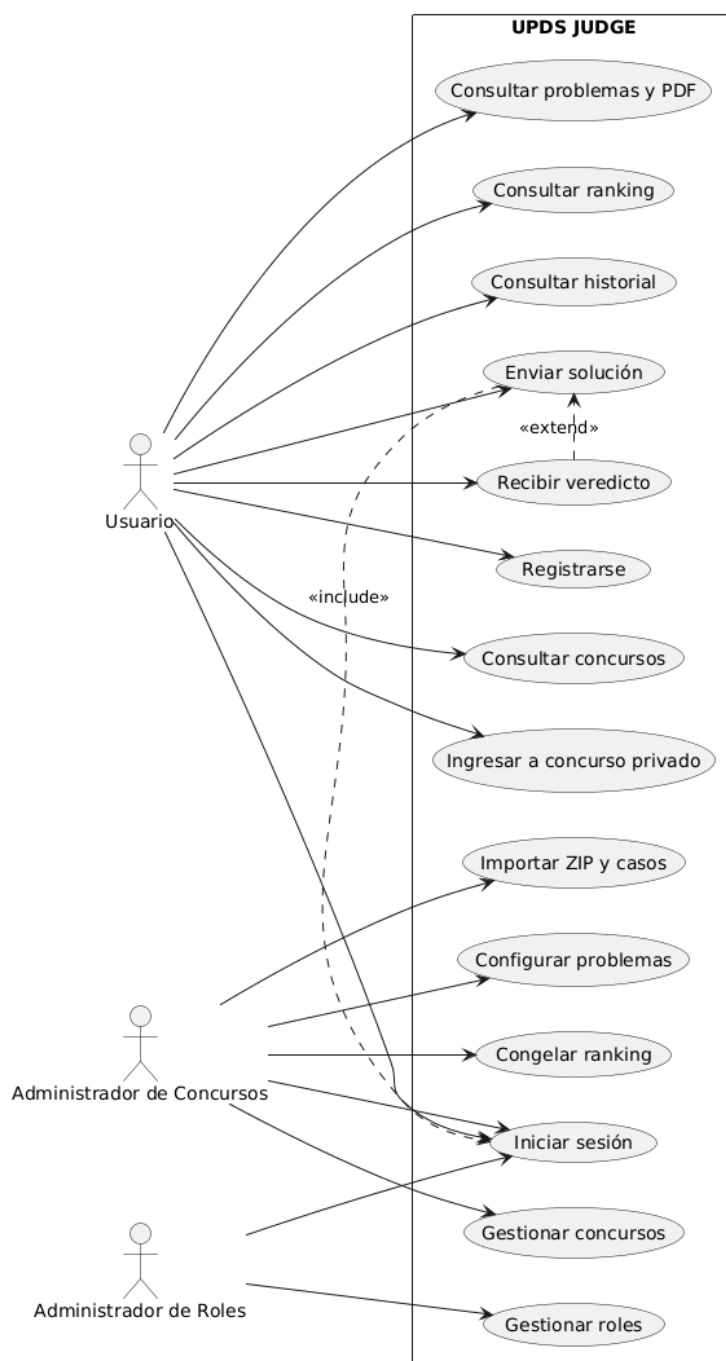


Figura 3: Diagrama general de casos de uso. Fuente: fuente PlantUML aprobada del proyecto.

5.2. Casos principales

El Administrador de Roles consulta usuarios y gestiona asignaciones autorizadas; el Administrador de Concursos crea o edita concursos, configura privacidad y duración, incorpora PDF y ZIP, administra problemas y consulta herramientas del concurso; y el Usuario se registra, inicia

sesión, se inscribe, consulta problemas, descarga el PDF, envía soluciones y revisa veredictos, historial, ranking y upsolving. Ningún caso de uso entregado permite editar el perfil personal.

5.3. Flujos básicos y alternativos

El flujo competitivo básico es autenticación, acceso al concurso, selección de problema y lenguaje, envío, evaluación y consulta del resultado. Entre los flujos alternativos se encuentran contraseña incorrecta, concurso no accesible, archivo inválido, juez temporalmente no disponible, payload contractual inválido y ranking congelado.

5.4. Relación entre requerimientos e historias

Las historias UJ-05 a UJ-07 cubren identidad; UJ-08 a UJ-10, administración; UJ-11 a UJ-13, participación; UJ-14 a UJ-16, evaluación; y UJ-18 a UJ-20, ranking e historial. El identificador UJ-17 no existe en la fuente original y no fue inventado.

Fase III

Diseño de la solución

6. Arquitectura

6.1. Visión general

La arquitectura separa la SPA, la API, la persistencia y el motor de evaluación. La Figura 6.1 muestra los componentes y sus dependencias aprobadas.

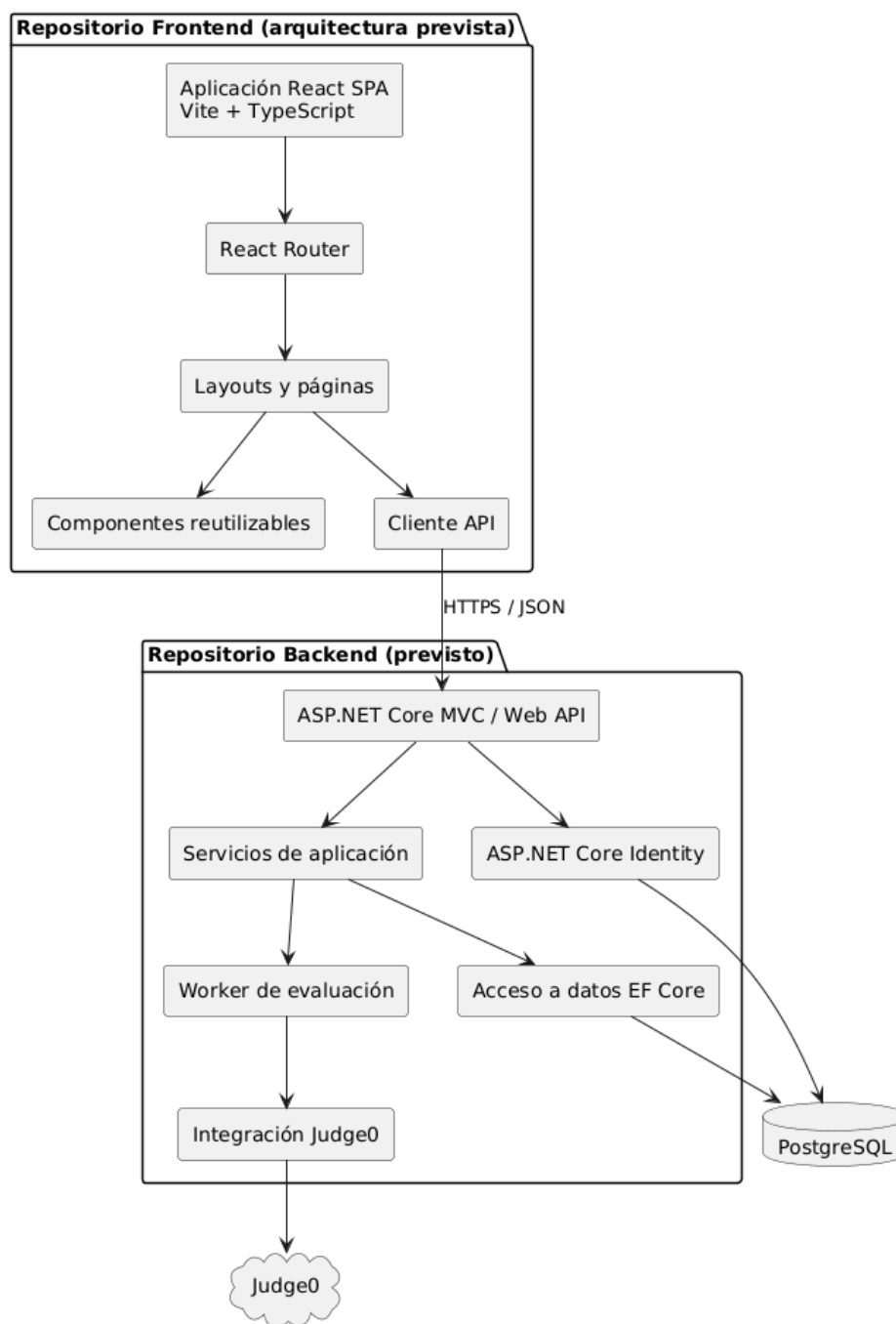


Figura 4: Arquitectura de componentes de UPDS JUDGE. Fuente: fuente PlantUML aprobada del proyecto.

6.2. Arquitectura frontend

El frontend se organiza por features de autenticación, concursos, problemas, envíos, historial, ranking y administración. React Router compone rutas públicas y protegidas; `UserLayout` y `AdminLayout` delimitan experiencias; TanStack Query coordina consultas y polling; el cliente HTTP incorpora la sesión; y los tipos generados reflejan el contrato consumido.

6.3. Backend

El backend documentado utiliza ASP.NET Core MVC/Web API, servicios de aplicación, Entity Framework Core e Identity. Expone contratos HTTP/JSON para autenticación, roles, concursos, participantes, envíos y ranking. Esta descripción se basa en la documentación técnica final aprobada por el equipo; no se realizó una auditoría adicional.

6.4. Motor de evaluación

El juez compila y ejecuta soluciones bajo límites de tiempo y memoria, normaliza estados y devuelve métricas. El backend encapsula sus credenciales y coordina la persistencia. El frontend recibe únicamente el contrato de resultados.

6.5. Comunicación entre componentes

La Figura 6.5 representa el flujo de envío, evaluación y persistencia. El frontend crea la solicitud, el backend valida identidad y reglas, el servicio de evaluación coordina el juez y la base de datos conserva el resultado.

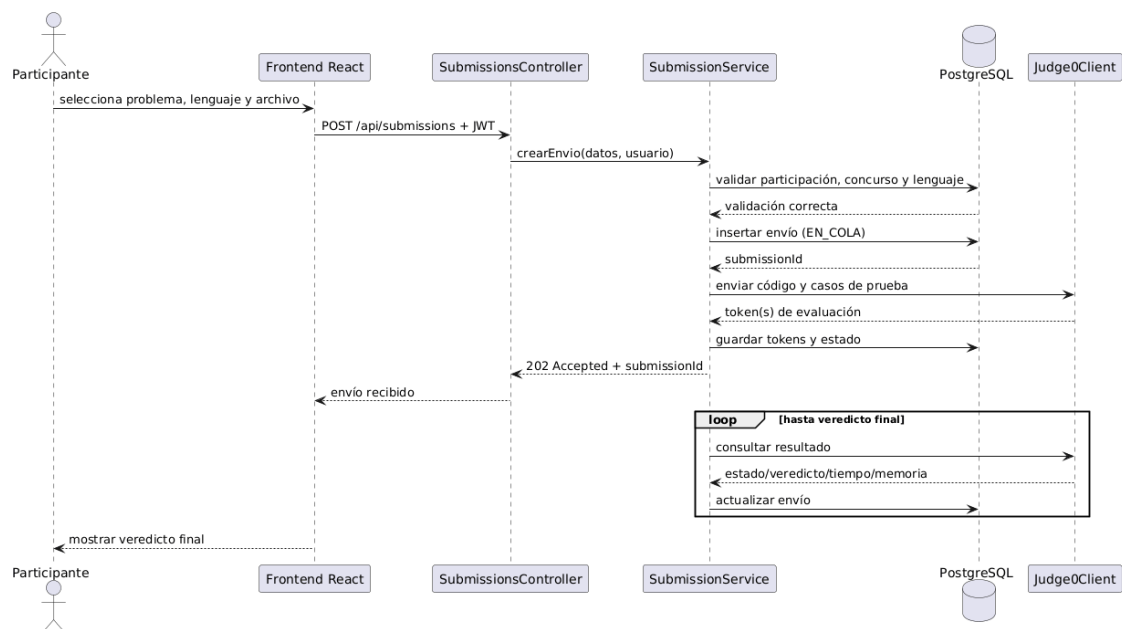


Figura 5: Secuencia de envío y evaluación de una solución. Fuente: fuente PlantUML aprobada del proyecto.

6.6. Seguridad y roles

JWT identifica la sesión y los roles; los guards evitan navegación incompatible; y el backend valida autorización. El Administrador de Roles no recibe autorización funcional irrestricta por su denominación; el Administrador de Concursos se limita a las capacidades administrativas documentadas; y el Usuario accede a funciones competitivas según inscripción, modalidad y estado del concurso. Los casos de prueba, código y credenciales no se exponen al navegador. La extracción ZIP requiere límites y prevención de rutas maliciosas.

7. Modelado

7.1. Diagrama de clases

La Figura 7.1 relaciona Usuario, Rol, Concurso, Participación, Problema, Caso de Prueba, Lenguaje, Envío y Resultado.

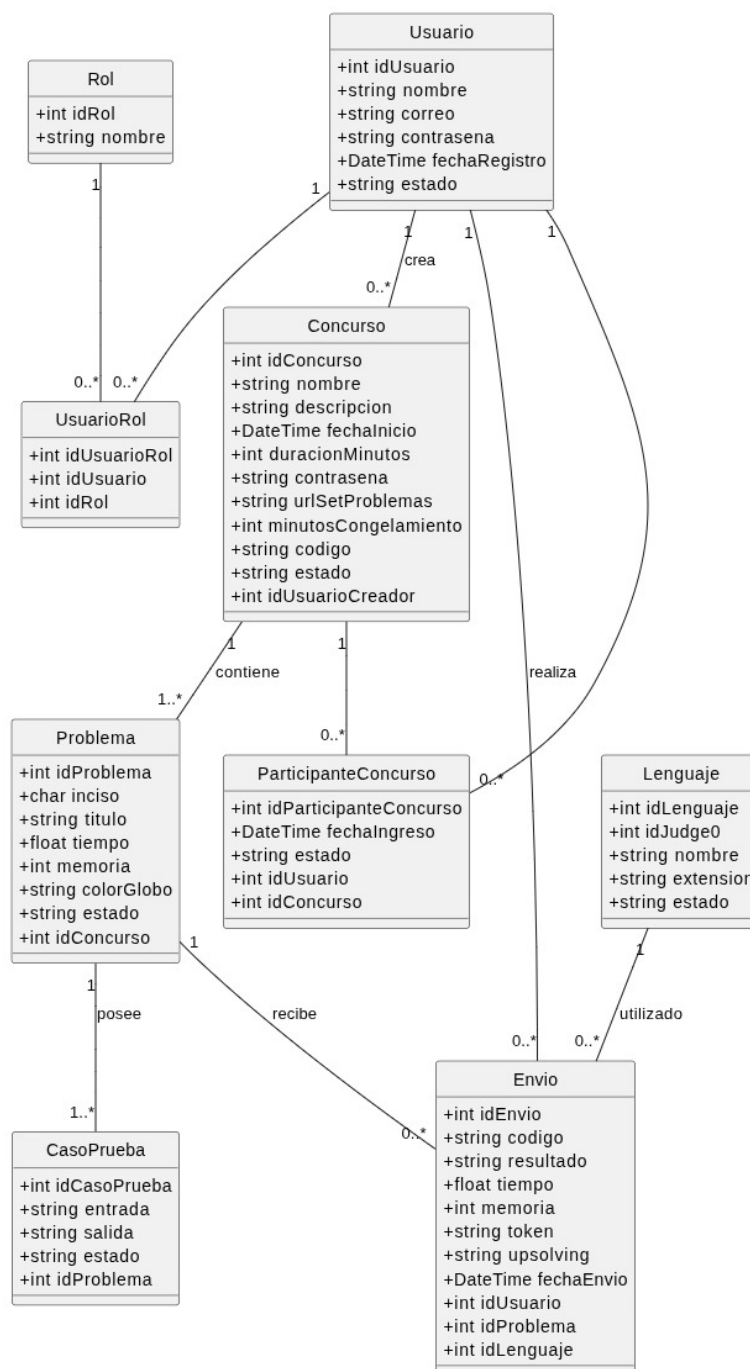


Figura 6: Diagrama de clases de persistencia. Fuente: fuente PlantUML aprobada del proyecto.

7.2. Modelo de contexto

La Figura 7.2 distingue frontend, backend, persistencia y juez. Cualquier conexión de tiempo real representada se interpreta como integración prevista, porque el frontend final no contiene un cliente SignalR o WebSocket conectado.

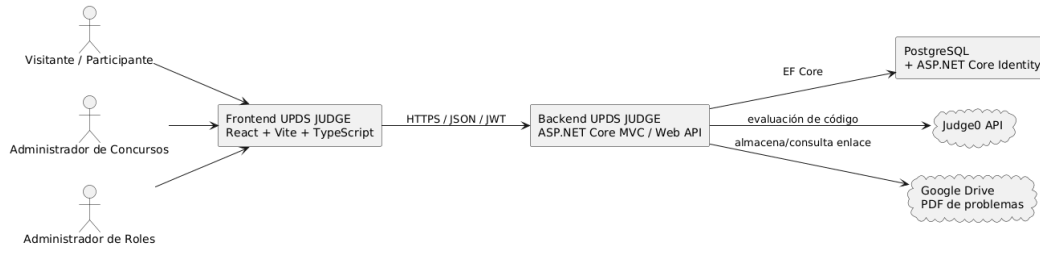


Figura 7: Modelo de contexto técnico y actores. Fuente: fuente PlantUML aprobada del proyecto.

7.3. Diagramas de secuencia o actividad

Además de la secuencia de envío, la Figura 7.3 documenta la importación del ZIP con validación previa, tratamiento transaccional y almacenamiento privado.

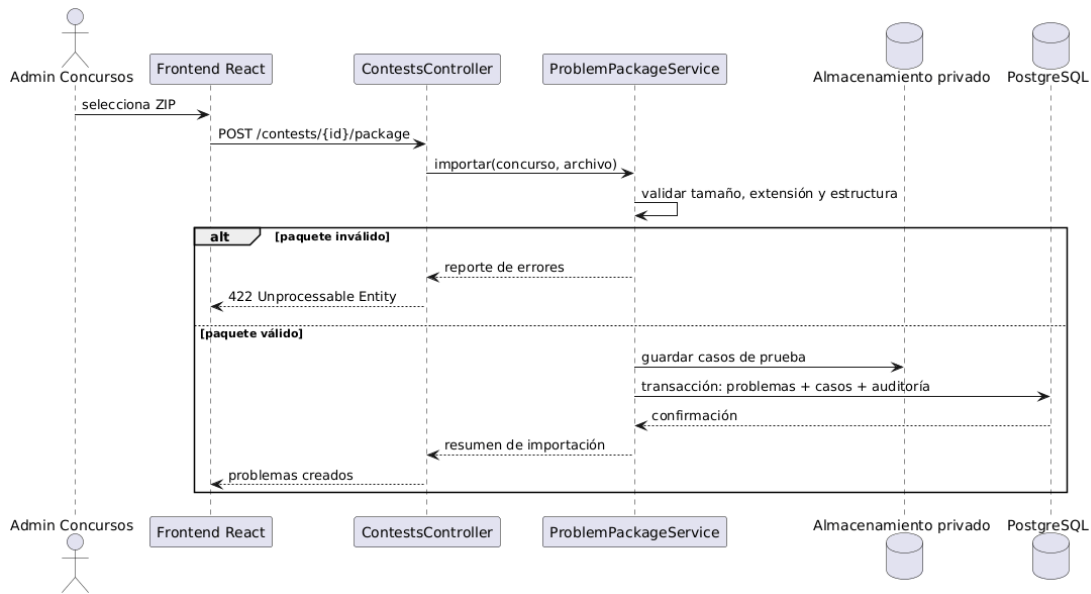


Figura 8: Secuencia de importación de problemas y casos desde ZIP. Fuente: fuente PlantUML aprobada del proyecto.

7.4. Modelo de datos

La Figura 7.4 presenta el modelo relacional aprobado. Identity conserva credenciales y roles; concursos y participaciones delimitan acceso; problemas y casos representan la competencia; y envíos y resultados sostienen trazabilidad y ranking.

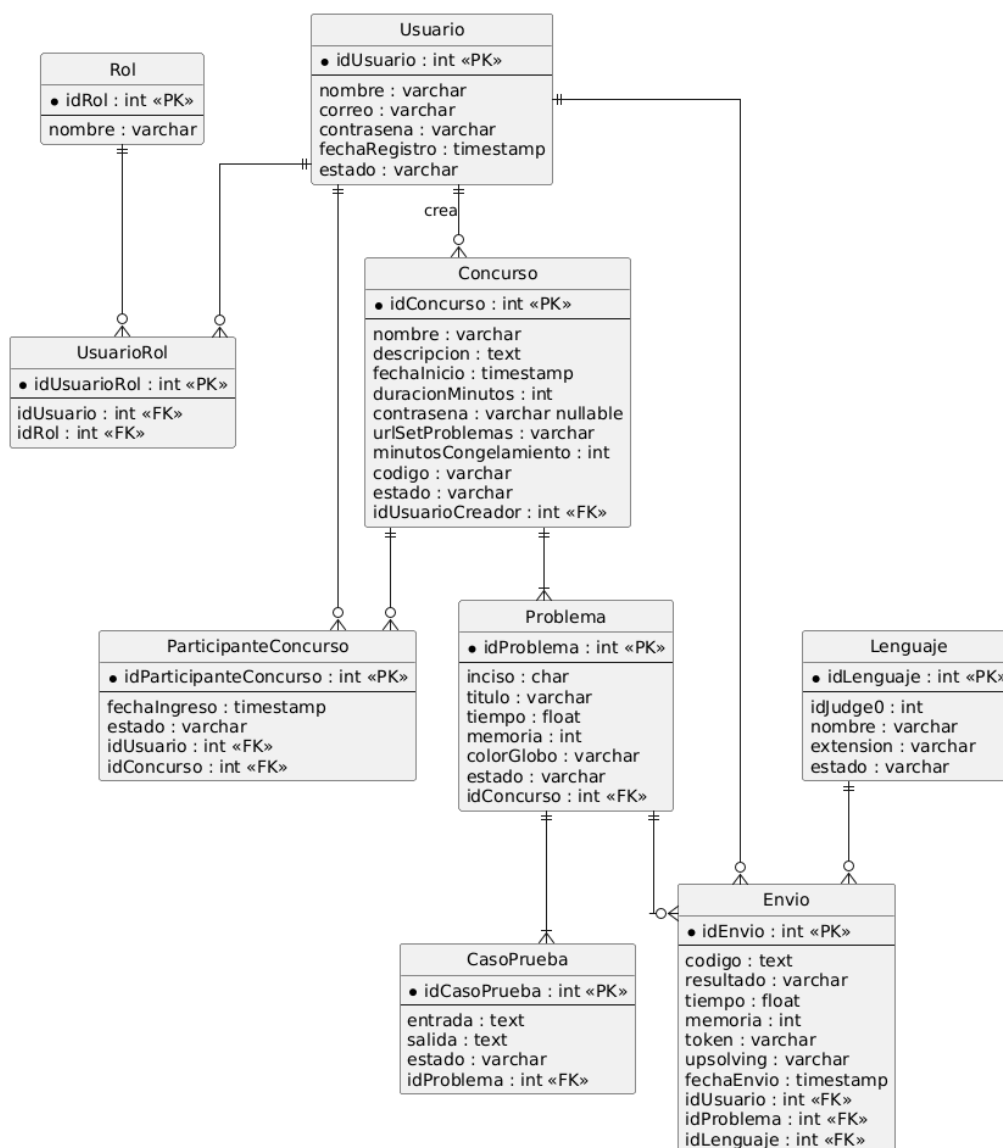


Figura 9: Modelo relacional de UPDS JUDGE. Fuente: fuente PlantUML aprobada del proyecto.

7.5. Diccionario de datos o referencia

Las entidades principales son Concurso, ParticipaciónConcurso, Problema, CasoPrueba, Lenguaje, Envío y ResultadoCaso. El diccionario detallado permanece en la arquitectura de datos aprobada; este informe evita duplicarlo y conserva la referencia documental en la bibliografía.

Fase IV

Gestión ágil

8. Metodología Scrum

8.1. Organización del equipo

El equipo trabajó de manera incremental con backlog, historias, criterios, changes, revisiones y retrospectivas. La responsabilidad funcional del producto permaneció separada de la coordinación del equipo y de las asignaciones de ejecución.

8.2. Roles Scrum

La documentación registra coordinación del squad, revisión, evidencias, pruebas, backend y assets. No existe evidencia suficiente para reinterpretar esas responsabilidades como una designación formal adicional; por ello se conservan únicamente las funciones documentadas.

8.3. Matriz RACI

La Figura 10 muestra la asignación de responsabilidades utilizada por el equipo. Se interpreta como instrumento de coordinación: identifica participación y responsabilidad por actividad sin confundirse con los permisos del sistema. No se expanden abreviaturas o identidades que no estén definidas en la fuente.

RACI MATRIZ								
Historias de Usuario	AF	JA	WY	CA	AT	BV	EL	
UJ-5	C	I	A	R	I	I	I	Alex Fernandez AF
UJ-6	C	R	A	I	I	I	I	Javier Aramayo JA
UJ-7	C	I	A	I	R	I	I	Wilson Yucra WY
UJ-8	C	I	A	I	R	I	I	Cristhian Amador CA
UJ-9	R	I	A	I	I	I	I	Arnold Torrez AT
UJ-10	R	I	A	I	I	I	I	Beymar Vasquez BV
UJ-11	C	I	A	I	R	I	I	
UJ-12	R	I	C	I	I	I	I	
UJ-13	A	I	A	R	I	I	I	Responsable R
UJ-14	A	R	C	I	I	I	I	Consultado C
UJ-15	A	R	C	I	I	I	I	Informado I
UJ-16	A	R	C	I	I	I	I	Aprobador A
UJ-18	R	I	A	I	I	I	I	
UJ-19	A	I	A	R	I	I	I	
UJ-20	I	R	C	I	I	I	I	
UJ-21	R	C	A	C	C	C	C	
UJ-22	R	C	A	C	C	C	C	
UJ-23	C	I	R	I	I	I	I	
UJ-24	C	I	R	I	I	I	I	
UJ-25	C	I	R	I	I	I	I	
UJ-26	C	I	R	I	I	I	I	
UJ-27	R	I	C	I	I	I	I	

Figura 10: Matriz RACI del proyecto. Fuente: elaboración del equipo del proyecto.

8.4. Product Backlog

El backlog contiene 15 historias y cinco épicas. La Tabla 6 resume su distribución sin repetir criterios completos.

Cuadro 6: Resumen del Product Backlog y estado final.

ID	Resumen	Sprint	Estado final	Requerimientos relacionados
UJ-05	Registro de usuario	1	Implementado	RF-01, RNF-04
UJ-06	Inicio de sesión y token con roles	1	Implementado	RF-01, RF-24
UJ-07	Asignar y revocar roles	3	Implementado	RF-17, RF-20
UJ-08	Crear y administrar concurso	1	Implementado	RF-21, RF-23
UJ-09	Importar paquete ZIP	1	Implementado	RF-15, RF-16, RF-25
UJ-10	Configurar problemas	2	Implementado	RF-15, RF-16
UJ-11	Listar concursos por estado	2	Implementado	RF-03, RF-04
UJ-12	Inscribirse a concursos	2	Implementado	RF-05, RF-24, RF-27
UJ-13	Consultar problemas y PDF	2	Implementado	RF-06, RF-07
UJ-14	Enviar solución	2	Implementado	RF-08, RF-09, RF-25
UJ-15	Recibir y consultar veredicto	2	Con limitaciones	RF-12, RF-19, RF-26
UJ-16	Practicar mediante upsolving	3	Implementado	RF-18
UJ-18	Consultar ranking	3	Implementado	RF-14
UJ-19	Configurar congelamiento	3	Implementado	RF-21, RF-23
UJ-20	Consultar historial global	3	Implementado	RF-13, RF-19

8.5. Product Goal

El Product Goal fue disponer de un flujo integrado y trazable de concursos de programación, desde identidad y configuración hasta evaluación y ranking. Su logro se evalúa en la Sección 14.

8.6. Sprint Backlogs y Sprint Goals

Sprint 0 preparó fundaciones; Sprint 1 buscó habilitar login y creación de concursos con ZIP; Sprint 2 concentró participación y juez; y Sprint 3 cerró ranking, congelamiento, historial y roles. La asignación se conserva según el plan documentado.

9. Ejecución de Sprints

9.1. Sprint 0

El Sprint 0 definió repositorios, ramas, estructura React/Vite/TypeScript, backend ASP.NET Core, persistencia, contratos, CORS, variables y componentes base. Su resultado fue una fábrica de software y documentación inicial para ejecutar historias incrementales.

9.2. Sprint 1

El Sprint 1 cubrió autenticación, shell, administración de concursos, creación e importación ZIP. La retrospectiva destacó la necesidad de contratos estables, boundaries de features, componentes reutilizables y validación reproducible. UJ-11 permanecía futura en ese momento, pero fue implementada posteriormente.

9.3. Sprint 2

El Sprint 2 consolidó participación, concursos privados, problemas, envíos y contexto reutilizable. Sus acuerdos posteriores incorporaron Acceso de Usuario bajo `AdminLayout`, la política de privado finalizado, remediación de dependencias y `ContestContextHeader`. La retrospectiva conserva la diferencia entre acuerdo histórico e implementación posterior.

9.4. Sprint 3

El Sprint 3 integró roles, acceso administrativo de usuario, edición de concursos, congelamiento, colores de globos, ranking, polling, paginación, historial y hotfixes. También registró dos propuestas futuras: versiones estables de lenguajes y actualización de datos personales.

9.5. Sprint Reviews

Las reviews se documentaron mediante criterios de aceptación, changes, pruebas y confirmaciones manuales existentes. Cuando una ceremonia no dispone de acta específica, este informe no inventa asistentes, decisiones ni fechas.

9.6. Retrospectivas

Las retrospectivas aplicaron la técnica Starfish y distinguieron prácticas a mantener, reducir, ampliar, detener e iniciar. Los aprendizajes centrales fueron validar contratos antes de integrar, mantener layouts como contenedores, usar datos contractuales y complementar pruebas unitarias con router y requests reales.

9.7. Acciones futuras no implementadas

Versiones estables de lenguajes: propuesta no implementada. El catálogo actual no demuestra una ampliación contractual ni nuevos runtimes.

Configuración y actualización de datos personales: propuesta no implementada. No existe ruta, pantalla, formulario o servicio de perfil editable.

Fase V

Planificación y control

10. Planificación temporal

10.1. Gantt general por Sprint

La Figura 11 representa la secuencia general de Sprints y respalda el seguimiento de las fases de fundación, administración, participación y cierre. La lectura horizontal permite relacionar

duración relativa y orden de ejecución sin introducir fechas adicionales a las consignadas en la fuente.

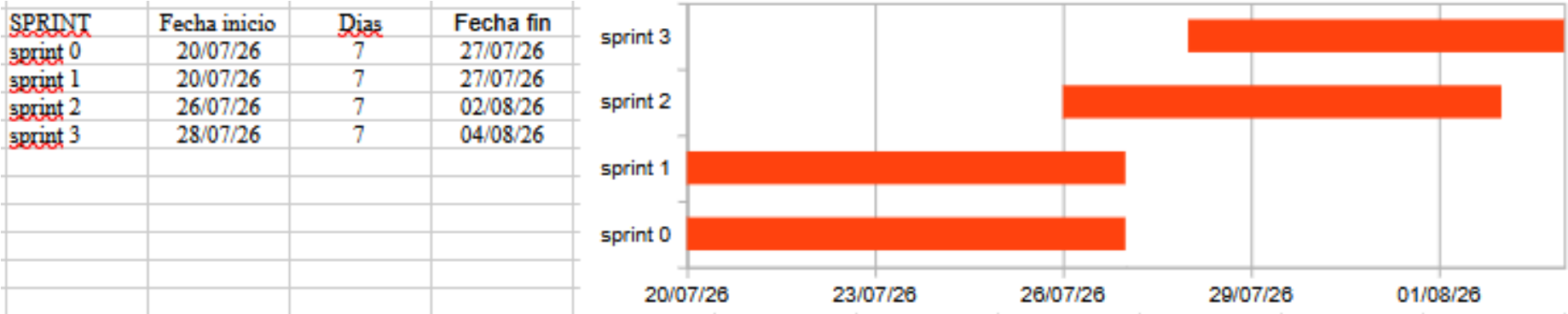


Figura 11: Diagrama de Gantt general por Sprint. Fuente: elaboración del equipo del proyecto.

10.2. Gantt de historias de usuario

La Figura 12 distribuye las historias dentro del calendario del proyecto. Su función es evidenciar dependencias y concentración de trabajo, especialmente entre identidad, concursos, juez y ranking.

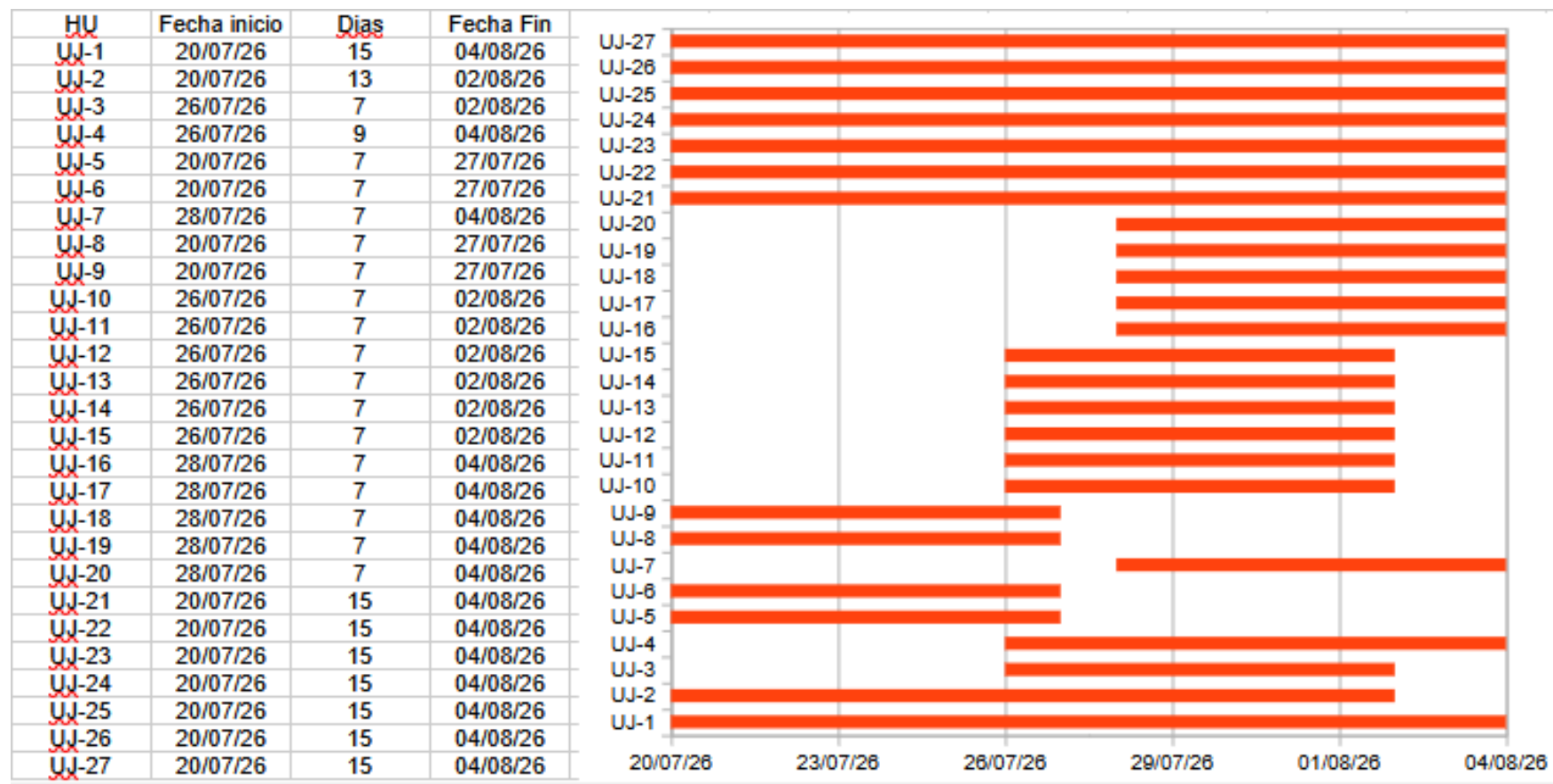


Figura 12: Distribución temporal de historias de usuario. Fuente: elaboración del equipo del proyecto.

10.3. Hitos y entregables

Los hitos fueron: fundaciones y contratos; autenticación y administración; participación y evaluación; ranking y cierre; y reconciliación documental. Los entregables comprenden el frontend, contratos consumidos, historias, diagramas, retrospectivas, evidencias existentes y este informe final.

11. Gestión de riesgos

11.1. Metodología

La gestión de riesgos identifica eventos, estima probabilidad e impacto, prioriza su tratamiento y registra mitigaciones. La prioridad orienta el seguimiento, pero no sustituye la validación técnica ni permite inventar valores ausentes.

11.2. Registro priorizado

La Figura 13 contiene el registro suministrado. Se utiliza para reconocer amenazas del producto y del proceso y vincularlas con acciones preventivas, sin transcribir valores que no puedan verificarse en la fuente.

#	Riesgo	Probabilidad	Impacto	Puntaje (P x I)	Prioridad
1	Código de un participante agota la memoria/CPU del servidor compartido	4 - Probable	5 - Catastrófica	20	Alta
2	Fallo de PostgreSQL bajo alta concurrencia de escrituras	4 - Probable	4 - Importante	16	Alta
3	Vulnerabilidad de inyección SQL u otra falla de seguridad en la API	3 - Posible	5 - Catastrófica	15	Alta
4	Condición de carrera al actualizar el ranking en tiempo real	4 - Probable	3 - Moderada	12	Alta
5	Bug en el cálculo del ranking o la penalización	3 - Posible	4 - Importante	12	Alta
6	Sobrecarga del servidor de evaluación en picos de envíos simultáneos	5 - Muy probable	2 - Menor	10	Alta
7	Código malicioso escapa del sandbox de ejecución	2 - No es probable	5 - Catastrófica	10	Alta
8	Veredictos TLE/MLE mal clasificados por límites de tiempo/memoria mal configurados	3 - Posible	3 - Moderada	9	Media
9	Copia de soluciones entre participantes	4 - Probable	2 - Menor	8	Media
10	Usuario obtiene el rol de Admin de forma indebida	2 - No es probable	4 - Importante	8	Media
11	Error al parsear el ZIP con la estructura de casos de prueba	3 - Posible	2 - Menor	6	Media
12	Filtración de los casos de prueba antes del concurso	1 - Muy improbable	5 - Catastrófica	5	Media
13	Pérdida de conexión del usuario al enviar su código	4 - Probable	1 - Insignificante	4	Baja
14	Incompatibilidad de versiones entre lenguajes soportados (C++, Python, C#)	2 - No es probable	2 - Menor	4	Baja
15	Pérdida de datos en PostgreSQL por falta de respaldos	1 - Muy improbable	4 - Importante	4	Baja
16	Retraso por curva de aprendizaje del equipo en ASP.NET Core / EF Core	3 - Posible	1 - Insignificante	3	Baja

Figura 13: Listado priorizado de riesgos del proyecto. Fuente: elaboración del equipo del proyecto.

11.3. Matriz de probabilidad e impacto

La Figura 14 relaciona probabilidad e impacto para determinar prioridad. Su lectura permite enfocar mitigaciones en ejecución de código no confiable, exposición de casos, disponibilidad del juez, contratos, ranking y planificación.

		Gravedad				
	1 - Insignificante	2 - Menor	3 - Moderada	4 - Importante	5 - Catastrófica	
Probabilidad	5 - Muy probable	5 Sobrecarga del servidor de evaluación en picos de envíos simultáneos	10 Sobrecarga del servidor de evaluación en picos de envíos simultáneos	15	20	25
	4 - Probable	4 Pérdida de conexión del usuario al enviar su código	8 Copia de soluciones entre participantes (fraude)	12 Condición de carrera al actualizar el ranking en tiempo real	16 Fallo de PostgreSQL bajo alta concurrencia de escrituras	20 Código de un participante agota la memoria/CPU del servidor compartido
	3 - Posible	3 Retraso por curva de aprendizaje del equipo en ASP.NET Core / EF Core	6 Error al parsear el ZIP con la estructura de casos de prueba	9 Veredictos TLE/MLE mal clasificados por límites de tiempo/memoria mal configurados	12 Bug en el cálculo del ranking o la penalización	15 Vulnerabilidad de inyección SQL u otra falla de seguridad en la API
	2 - No es probable	2 Incompatibilidad de versiones entre lenguajes soportados (C++, Python, C#)	4 Incompatibilidad de versiones entre lenguajes soportados (C++, Python, C#)	6	8 Usuario obtiene el rol de Admin de forma indebida	10 Código malicioso escapa del sandbox de ejecución
	1 - Muy improbable	1	2	3	4 Pérdida de datos en PostgreSQL por falta de respaldos	5 Filtración de los casos de prueba antes del concurso

Figura 14: Matriz de probabilidad, impacto y prioridad. Fuente: elaboración del equipo del proyecto.

11.4. Medidas de mitigación

Las medidas documentadas incluyen aislamiento en el juez, límites de archivos, almacenamiento privado, autorización, contratos versionados, mappers tipados, pruebas de ranking con datos conocidos, polling acotado, paginación y validación de rutas. Los hotfixes del ranking muestran la aplicación concreta de diagnóstico causal y pruebas de regresión.

11.5. Riesgos residuales

Permanecen riesgos de dependencia de servicios externos, medición de carga, ausencia de push en tiempo real, deuda de pruebas backend señalada por la retrospectiva y desalineación futura de contratos. Se recomienda tratarlos mediante trabajo específico, sin clasificarlos como fallas actuales del backend o juez.

Fase VI

Implementación, validación y cierre

12. Implementación final

12.1. Tecnologías

El frontend utiliza React, TypeScript, Vite, React Router, TanStack Query, React Hook Form, Zod, Tailwind CSS, Vitest, Testing Library y MSW. La solución documentada utiliza ASP.NET Core MVC/Web API, Entity Framework Core, PostgreSQL, JWT y un motor de evaluación externo. Las versiones precisas se mantienen en los manifiestos; el informe evita convertirlas en decisiones académicas permanentes.

12.2. Módulos principales

Los módulos se organizan en autenticación, administración, concursos, problemas, submissions, historial y ranking. Componentes compartidos proporcionan branding, navegación, formularios y tablas; la librería común concentra API, autenticación, queries y utilidades.

12.3. Autenticación y autorización

Registro y login consumen contratos de Auth. La sesión se conserva en `sessionStorage`; la identidad y roles se derivan del token; y `ProtectedRoute` y `RoleRoute` controlan el acceso visible. La seguridad definitiva sigue en el backend.

12.4. Concursos

El usuario consulta concursos filtrados y puede inscribirse en modalidades públicas o privadas. El administrador dispone de listado, resumen, filtros, creación, ZIP, edición, privacidad, fechas, duración y minutos de congelamiento. Los concursos privados finalizados aplican la política contractual documentada.

12.5. Problemas

Las vistas de problemas muestran incisos, títulos, límites, PDF y estado personal normalizado como `ACCEPTED`, `UNSOLVED`, `NOT ATTEMPTED` o neutral desconocido. La administración configura problemas y casos mediante el concurso y su ZIP.

12.6. Envíos

El participante selecciona problema y lenguaje, escribe o carga código, valida extensión y tamaño y envía texto fuente al backend. Se admiten C++ 17, Python 3 y C# en el selector documentado; no se afirma una ampliación posterior. El resultado se consulta por HTTP y se presenta con veredicto, tiempo y memoria.

12.7. Ranking

El backend ordena por problemas resueltos, menor penalización y criterio determinista, aplicando veinte minutos por intento fallido previo al primer Accepted. Upsolving se excluye. El frontend conserva puestos, usa polling de diez segundos, muestra countdown, paginación, congelamiento y globos, y ofrece flujo normal y administrativo. El hotfix corrigió datos temporales del entorno, preservación del payload, registro de la ruta administrativa y una prop privada enviada al DOM; esas regresiones no se presentan como defectos vigentes.

12.8. Administración

El AdminLayout ofrece navegación por rol. El Administrador de Roles —docente o líder de club— consulta usuarios y administra asignaciones permitidas, incluida la delegación de Administración de Concursos, sin recibir acceso irrestricto a toda función. El Administrador de Concursos —docente u organizador— crea y edita concursos, configura privacidad y duración, incorpora PDF y ZIP, administra problemas y consulta herramientas administrativas. Acceso de Usuario reutiliza concursos, Mis Envíos, problemas, submissions y ranking sin anidar UserLayout.

12.9. Integración frontend

Los endpoints se centralizan y los builders codifican códigos de concurso. TanStack Query gestiona caché e invalidación; mappers adaptan contratos; y los estados de carga, vacío y error se presentan en la interfaz. La carpeta de realtime no contiene cliente conectado, por lo que no se atribuye push a la entrega final.

12.10. Despliegue y reproducibilidad

La operación conceptual respeta el orden de dependencias: primero se levanta Judge0 CE y su entorno Docker; después, el backend y su persistencia; finalmente, el frontend. El cliente se instala y ejecuta desde `frontend/`, consume un prefijo HTTP relativo y requiere que el hosting preserve el fallback de la SPA hacia `index.html` sin interceptar el prefijo de API. Los comandos, variables y puertos completos se mantienen en las guías enlazadas; este informe evita duplicarlas.

La `guia.md` aprobada documenta Judge0 CE v1.13.1, la imagen publicada del backend, PostgreSQL y el flujo local del frontend. Se utilizó como fuente de solo lectura y se contrastó con `package.json`, Vite, variables confirmadas, rutas y documentación reconciliada. No se copiaron contraseñas, claves ni otros valores sensibles de sus ejemplos.

12.11. Recursos y evidencias digitales del proyecto

La Tabla 7 centraliza los enlaces autorizados. El repositorio backend y el juez se enlazan como servicios externos asumidos correctos; su inclusión no implica una auditoría de código o ejecución.

Cuadro 7: Recursos y evidencias digitales del proyecto.

Recurso	Propósito	Ubicación
Repositorio frontend	Código y documentación del cliente React/Vite.	Repositorio frontend de UPDS JUDGE
Repositorio backend	API, persistencia, autorización e integración con el juez.	Repositorio backend de UPDS JUDGE
Guía de instalación backend	Instalación y configuración mantenidas por el repositorio backend.	Consultar guía del backend
Entorno del juez Docker	Recursos versionados para levantar Judge0 CE v1.13.1.	Consultar entorno y guía de Judge0
Imagen del backend Docker	Imagen publicada identificada en la guía aprobada.	Consultar imagen Docker del backend
Audio de la entrevista	Evidencia autorizada del levantamiento de requerimientos.	Escuchar audio de la entrevista
Guía del frontend	Arquitectura, convenciones, instalación y desarrollo del cliente.	Consultar guía técnica del frontend
README del frontend	Configuración, build, preview, despliegue y servicios relacionados.	Consultar guía operativa del frontend
Auditoría aprobada	Estado reconciliado y evidencias del frontend.	Consultar auditoría documental

13. Calidad y validación

13.1. Estrategia de pruebas

La calidad combinó pruebas unitarias, de componentes, servicios, mappers, layouts y router; validación manual existente; lint; typecheck; build; auditoría de dependencias y revisión de contratos. Las pruebas de ranking y rutas se ampliaron ante regresiones reales.

13.2. Pruebas automatizadas

Las cantidades pertenecen a cierres específicos: UJ-08/UJ-09 registró 41 pruebas; UJ-11, 84; el change de acciones de Sprint 2, 105; UJ-19, 116; y el hotfix UJ-18, 144. Durante la auditoría documental del 28/07/2026 se ejecutaron 34 archivos y 145 pruebas aprobadas. Ninguna cifra se presenta como conteo permanente del proyecto.

13.3. Validaciones manuales

Las capturas existentes cubren autenticación, roles, concursos, problemas, envíos, ranking, globos e historial. Para ranking normal y administrativo existe una validación manual confirmada por el responsable del proyecto. La ausencia de una captura nueva no se interpreta como funcionalidad pendiente.

13.4. Incidencias y hotfixes

Los incidentes relevantes incluyeron payload de ranking antiguo, seed temporal fuera del corte, ruta administrativa no registrada y una prop privada de formulario. Los fixes corrigieron normalización, datos, router y semántica DOM; además se mantuvieron pruebas de regresión de polling, congelamiento, countdown y paginación.

13.5. Limitaciones

- RF-02 no fue implementado.
- RF-22 se cumplió parcialmente por no incluir edición del perfil.
- El frontend no posee cliente SignalR o WebSocket; consulta resultados por HTTP.
- Las garantías de carga, disponibilidad y concurrencia no cuentan con métricas finales cuantitativas en la documentación aprobada.
- La ampliación de versiones o cantidad de lenguajes no fue implementada.

13.6. Trazabilidad final

La Tabla 5 vincula los 46 requerimientos iniciales con evidencia, historias y observaciones. Los estados se basan en la documentación reconciliada, no en una reescritura retroactiva de la intención inicial.

14. Resultados

14.1. Objetivos alcanzados

Se alcanzó el objetivo general al disponer de un flujo integrado de concursos, evaluación y resultados, respaldado por identidad, roles, persistencia y arquitectura separada. También se consolidaron navegación contextual, administración, historial, ranking, congelamiento, globos y upsolving.

14.2. Requerimientos cumplidos

La mayoría de los RF fue implementada; los RNF cuentan con evidencia arquitectónica y funcional, aunque algunos conservan limitaciones de medición o dependencia externa. La trazabilidad evita equiparar evidencia documental con garantía cuantitativa.

14.3. Requerimientos parciales

RF-22 es **IMPLEMENTADO PARCIALMENTE** porque agrupa capacidades entregadas y la edición del perfil no implementada. RF-12 y RF-26 se consideran implementados con limitaciones porque el resultado y el estado se consultan por HTTP, sin push conectado.

14.4. Requerimientos no implementados

RF-02 es **NO IMPLEMENTADO**. La ausencia se limita a gestión editable del perfil y no invalida el valor global de la plataforma. No existe pantalla, configuración, formulario, guardado ni actualización de sesión por edición de datos personales.

14.5. Trabajo futuro

1. Implementar gestión y edición segura del perfil del usuario.
2. Definir y ampliar versiones estables de lenguajes, con catálogo contractual y pruebas por runtime. Esta propuesta no está implementada.
3. Fortalecer pruebas backend cuando corresponda, como trabajo específico.
4. Mantener sincronizados contratos, tipos generados y documentación.
5. Automatizar validaciones documentales y LaTeX cuando sea viable.
6. Evaluar push en tiempo real sin eliminar la recuperación HTTP.

15. Conclusiones y recomendaciones

UPDS JUDGE alcanzó su objetivo general: centraliza la operación de concursos académicos y ofrece valor mediante evaluación automatizada, trazabilidad de envíos, reglas uniformes de ranking y separación de responsabilidades. La arquitectura facilita evolución y mantenimiento; las historias, changes, pruebas y evidencias muestran una ejecución incremental con correcciones de integración.

Las capacidades principales entregadas abarcan autenticación, roles, concursos, inscripción, problemas, PDF, envíos, veredictos, historial, ranking, penalización, congelamiento, globos, upsolving y administración. El frontend usa los contratos aprobados del backend y juez, cuya corrección se asume en este informe.

Las desviaciones se declaran de forma explícita: **RF-02 no fue implementado** y **RF-22 se cumplió parcialmente**. La gestión de perfil debe priorizarse como una capacidad futura independiente. De igual manera, las versiones estables y la ampliación de lenguajes constituyen una propuesta no implementada, no un resultado del Sprint 3.

Se recomienda conservar la disciplina de contratos y regresión, ampliar validaciones operativas cuantitativas, fortalecer pruebas backend cuando corresponda, automatizar compilación documental y mantener una revisión manual antes de cerrar este change.

A. Glosario

API

Interfaz de programación de aplicaciones utilizada para la comunicación HTTP entre frontend y backend.

JWT

JSON Web Token utilizado para representar identidad y roles de la sesión.

SPA

Aplicación de una sola página cuya navegación se resuelve en el cliente.

Scrum

Marco de trabajo iterativo basado en Product Backlog, Sprints, revisión y retrospectiva.

Sprint

Iteración acotada en la que se persigue un objetivo y se entrega un incremento.

Product Backlog

Lista priorizada de necesidades e historias del producto.

Sprint Backlog

Selección y plan de trabajo de un Sprint.

UPDS JUDGE

Plataforma académica de concursos y evaluación de programación.

Juez en línea

Servicio que compila y ejecuta soluciones bajo límites controlados.

Veredicto

Resultado normalizado de la evaluación de un envío.

Accepted

Solución que satisface los casos de prueba aplicables.

Wrong Answer

Solución que produce una salida incorrecta.

Compilation Error

Solución que no puede compilarse.

Runtime Error

Solución que falla durante la ejecución.

Time Limit Exceeded

Solución que excede el tiempo permitido.

Memory Limit Exceeded

Solución que excede la memoria permitida.

Upsolving

Envío de práctica posterior que no modifica el ranking oficial.

Congelamiento

Corte temporal que oculta resultados recientes en el ranking público.

Penalización

Tiempo acumulado del ranking, incluyendo intentos fallidos previos según la regla documentada.

Administrador de Roles

Actor funcional, normalmente docente o líder de club, que consulta usuarios y administra roles dentro de los permisos definidos.

Administrador de Concursos

Actor funcional, docente u organizador, responsable de configurar y administrar la dimensión académica y competitiva de los concursos.

Usuario

Estudiante o participante que compite, practica y consulta sus resultados; no incluye edición de información personal.

RACI

Matriz de asignación de responsabilidades del equipo del proyecto; no representa los roles funcionales de autorización.

B. Matriz de trazabilidad ampliada

La matriz canónica completa se encuentra en la Tabla 5; no se duplica en anexos. El desglose adicional de RF-22 aparece en la Tabla 4.

C. Historias de usuario

Las historias completas se conservan bajo `docs/historias/`. La Tabla 6 ofrece el índice resumido, Sprint, estado y relación con requerimientos.

D. Diagramas

Los diagramas incluidos provienen de PNG aprobados y se relacionan con fuentes PlantUML existentes. Durante la elaboración del informe no se regeneraron ni modificaron dichas fuentes.

E. Tablas de planificación

Los cronogramas canónicos se encuentran en las Figuras 11 y 12; la matriz de responsabilidades aparece en la Figura 10.

F. Gestión de riesgos

El registro y la matriz de prioridad se presentan en las Figuras 13 y 14. Su interpretación se desarrolla en el Capítulo 11.

G. Referencias documentales

Referencias

- [1] Equipo UPDS JUDGE. *Auditoría de documentación del frontend — estado actual*. Documento aprobado, 28 de julio de 2026.
- [2] Equipo UPDS JUDGE. *README del repositorio UPDS-JUDGE-FRONT*. Estado reconciliado del frontend.
- [3] Equipo UPDS JUDGE. *Product Backlog*. Documento interno docs/03-product-backlog.md.
- [4] Equipo UPDS JUDGE. *Modelado UML*. Documento interno docs/05-modelado-uml.md.
- [5] Equipo UPDS JUDGE. *Arquitectura de datos*. Documento interno docs/06-arquitectura-datos.md.
- [6] Equipo UPDS JUDGE. *Pivot & Plan / Ready to Sprint*. Documento interno docs/07-plan-ready-to-sprint.md.
- [7] Equipo UPDS JUDGE. *Sprint 0 — Preparando la fábrica de software*. Documento interno docs/08-sprint-0-fabrica-software.md.
- [8] Equipo UPDS JUDGE. *Retrospectivas de los Sprints 1, 2 y 3*. Documentos internos aprobados.
- [9] Equipo UPDS JUDGE. *Changes OpenSpec, historias reconciliadas y hotfixes de UPDS JUDGE*. Repositorio documental del proyecto.
- [10] Equipo UPDS JUDGE. *Guía de despliegue UPDSJudge*. Archivo interno aprobado guia.md, consultado en modo de solo lectura.